DarkSage

API Specification

Version 0.2.1 — Draft

Owner: TheSinnerMan

Classification: Internal

Wisdom Over Noise

Source repository: LightSwornPVP/DarkSage
Source baseline commit: 34a65818d48f596122281075e18c6e5f36ec93b5
Generation date: 2026-07-25T13:31:10-04:00
Document ID: DS-006

Document Control

Document ID	DS-006
Title	API Specification
Version	0.2.1
Status	Draft
Owner	TheSinnerMan
Classification	Internal
Repository	LightSwornPVP/DarkSage
Created	2026-07-24
Last Updated	2026-07-24

Status lifecycle: Draft → Under Review → Approved → Superseded/Deprecated.

Revision History

Version	Date	Author	Summary
0.1.0	2026-07-24	TheSinnerMan	First controlled draft. Defines the API contract that realizes DS-ARC-001's client/server boundary: cross-cutting conventions (auth, versioning, pagination, errors, rate limiting, streaming, auditability) and 17 API domains covering every DS-002 requirement family and DS-005 entity, including the Transaction/Alert/Notification entities added in the DS-005 v0.2.0 repair. Authored using `.ai-workflow/DS-006-PREP-NOTES.md` as input.
0.2.0	2026-07-24	TheSinnerMan	Targeted repair pass for four independent-audit High findings: (H1) added DS-API-COR-010 defining a mechanism-agnostic session lifecycle contract (login, logout, refresh, revoke-one, revoke-all, device/session visibility) without choosing a token scheme; (H2) split DS-API-WKS-001 so the Committed/MVP baseline workspace no longer depends on the Planned WorkspaceLayout entity (DS-DB-017), backing it with ephemeral session-scoped state instead, and clarified DS-API-WKS-002 as the sole Planned owner of persisted layouts; (H3) added DS-API-EXE-005/006 defining a Planned Trade Proposal creation and pipeline-submission contract, traced to DS-EXE-002, with explicit two-step separation from order placement; (H4) replaced ambiguous combined method declarations with deterministic per-operation method/path/params/body/response/status contracts, in full for Scanner, Integration Credential, Alert, and Notification endpoint groups and for candle range/timeframe/filtering, and in compact form elsewhere. Also clarified that external broker/data-provider APIs remain behind DS-004 adapters, outside this client/backend contract, and added a defense-in-depth redaction note to the audit-log endpoint.
0.2.1	2026-07-24	TheSinnerMan	Focused H1-only repair: DS-API-COR-010 referenced `GET /auth/sessions/current` as the discovery point for refresh support without defining that operation. Added its full deterministic contract (purpose, auth requirement, empty request parameters, response schema including `refresh_supported`/`refresh_expires_at`, success/error statuses, no-secret-exposure guarantee) and distinguished it from `DELETE /auth/sessions/current` (read vs. revoke, same path). Token mechanism (JWT vs. opaque) remains unresolved per Appendix A Open Question #2. No other requirement modified.

Generated from DS-006 v0.2.1 — baseline 34a65818d48f596122281075e18c6e5f36ec93b5

Table of Contents

Generated from DS-006 v0.2.1 — baseline 34a65818d48f596122281075e18c6e5f36ec93b5

1. Purpose

DS-006 defines the API contract through which desktop and (later) mobile clients interact with the backend, per DS-ARC-001's committed client/server topology. It does not invent architecture (DS-004's concern) or entity schema (DS-005's concern); it defines the request/response contract that exposes DS-005's entities according to DS-002's requirement boundaries — what is callable, by whom, in what shape, and under what Release Classification.

2. Scope

Covers: cross-cutting API conventions (protocol, versioning, authentication, pagination, error format, rate limiting, real-time channels, auditability) and endpoint-group contracts for 17 domains: Market Data & Symbol, Scanner, Signal, Chart, Workspace, Integration & Credential, User & Preferences, Sage/AI, Strategy, Backtest, Strategy Performance, Risk, Portfolio, Execution/Auto-Trader/Broker, Alerts & Notifications, Education & Knowledge, and Audit & Observability.

Does not cover: database schema (DS-005), architectural component design (DS-004), UI data-binding (DS-007), or exact OpenAPI/DDL artifacts (implementation detail — this document states the normative contract, not generated code). External broker and market-data-provider APIs remain entirely behind the Broker Adapter (DS-EXE-006) and market-data-provider abstraction (DS-ARC-006) DS-004 owns; this document defines only the client-to-backend contract, never a provider-facing or broker-facing contract.

3. Audience

Backend/client contributors, independent auditors.

4. Definitions

See DS-001 §24, DS-002 §4, DS-004 §4, DS-005 §4. Additional term:

Term	Meaning
Endpoint group	A cohesive set of API operations serving one DS-002 requirement family / DS-005 entity domain, documented together under one `DS-API-<DOMAIN>` prefix

5. API Design Principles

DS-API-COR-001 — Backend-Authoritative REST API

Release Classification: Committed / MVP | **Governing Source:** DS-ARC-001 (Committed); DS-ARC-004 (Committed, FastAPI)

Description: DarkSage shall expose backend capability through a REST API served by the FastAPI application (DS-ARC-004), using JSON request/response bodies and OpenAPI-described

operations. Clients (desktop, and later mobile) shall access all trading-relevant and material product state exclusively through this API — never by reading the database directly or duplicating backend logic (DS-ARC-001, DS-ARC-019).

Acceptance Criteria:

- No client code path reads application state from a source other than this API.
- Every Committed/MVP endpoint is described in the backend's OpenAPI schema.

Testing: API-schema completeness check against this document's Committed/MVP endpoint list.

DS-API-COR-002 — Request/Response Envelope, Timestamps, and Data State

Release Classification: Committed / MVP | **Governing Source:** DS-PRD-008 (Committed, Data State Visibility)

Description: All timestamps in API responses shall be ISO-8601 UTC. Any response field carrying a DS-005 entity with a `data_state`/state-relevant concept (e.g., Quote's `data_state`, Position's mark-to-market source) shall surface that state and, where applicable, the source timestamp, per DS-PRD-008 — a client shall never receive a value without the means to tell whether it is current, delayed, stale, historical, or simulated.

Acceptance Criteria:

- Every endpoint returning a DS-PRD-008-relevant value includes its state and timestamp in the response body, not only in a separate metadata call.

Testing: Response-schema audit confirming state/timestamp presence on all data-state-relevant fields.

DS-API-COR-003 — API Versioning

Release Classification: Committed / MVP (obligation) | **Governing Source:** Baseline necessity — no product decision exists yet on scheme (see Appendix A)

Description: The API shall be versioned so that a breaking change does not silently break existing clients. The specific scheme (URL-path e.g. `/api/v1/`, header-based, or content-negotiation) is not yet decided — see Appendix A Open Question #1.

Acceptance Criteria:

- Every Committed/MVP endpoint is reachable through a versioned path/contract, whatever scheme is chosen.
- A breaking change to a Committed/MVP endpoint's contract requires a new version, not an in-place silent change.

Testing: Not yet applicable pending scheme decision; the obligation (versioning must exist) is testable once implemented.

DS-API-COR-004 — Authentication and Authorization

Release Classification: Committed / MVP | **Governing Source:** `SECURITY_RULES.md` "Authentication and Authorization" ("All non-public backend endpoints must require authentication... Authorization must be enforced separately from authentication")

Generated from DS-006 v0.2.1 — baseline 34a65818d48f596122281075e18c6e5f36ec93b5

Description: All non-public API endpoints shall require authentication, enforced on the backend (never trusting a client-side check alone, per `SECURITY_RULES.md` "Backend Enforcement"). Authorization shall be evaluated separately from authentication, against permission groups appropriate to the endpoint's sensitivity (at minimum: read-only, trade approval, Auto-Trader control, administrative settings, live-trading management, per `SECURITY_RULES.md`). The specific authentication scheme (JWT, opaque session token, OAuth2) is not yet decided — see Appendix A Open Question #2. Session issuance, renewal, visibility, and revocation are governed by DS-API-COR-010, independent of that unresolved choice.

Acceptance Criteria:

- No non-public endpoint is reachable without a valid, backend-verified authentication credential.
- An authenticated-but-unauthorized request receives a distinct response (403) from an unauthenticated one (401).
- High-risk actions (live-trading changes, Emergency Flatten, credential changes) require the strong-authentication tier `SECURITY_RULES.md` specifies, once those endpoints exist (Phase 7+/13).

Testing: Authentication/authorization test matrix per permission group; unauthenticated/unauthorized request rejection test.

DS-API-COR-005 — Pagination and Filtering

Release Classification: Committed / MVP (obligation) | **Governing Source:** Baseline necessity — no product decision exists yet on scheme (see Appendix A)

Description: Every list-returning endpoint (Scanner results, Signals, Watchlist items, Alerts, Notifications, Audit log queries) shall use a consistent pagination convention and support filtering appropriate to its domain. The specific pagination style (cursor-based vs. offset/limit) is not yet decided — see Appendix A Open Question #3.

Acceptance Criteria:

- No list endpoint returns an unbounded result set by default.
- Pagination metadata (or cursor) is present in every paginated response.

Testing: Pagination-boundary test (empty, single-page, multi-page) once a scheme is chosen.

DS-API-COR-006 — Error Response Schema

Release Classification: Committed / MVP | **Governing Source:** DS-OPS-003 (Committed, Understandable Error Handling)

Description: Every error response shall use a consistent, structured schema (at minimum: an error code, a plain-language message suitable for direct display per DS-OPS-003, and an optional detail object for expandable technical information) rather than an ad hoc per-endpoint shape.

Acceptance Criteria:

- Every Committed/MVP endpoint's error responses conform to the shared error schema.
- The plain-language `message` field never exposes a raw stack trace or internal exception text (DS-OPS-003).

Testing: Error-schema conformance test across a fixture set of induced failures per endpoint group.

DS-API-COR-007 — Rate Limiting

Release Classification: Planned | **Governing Source:** `SECURITY_RULES.md` "Security Testing" (names rate-limit tests as a category; defines no policy)

Description: The API should apply rate limiting to reduce abuse risk, particularly on authentication and high-cost endpoints (scan execution, backtest execution). The specific policy (limits, windows, per-user vs. per-IP) is not yet decided — see Appendix A Open Question #4.

Acceptance Criteria: Deferred pending policy decision.

Testing: Not yet applicable.

DS-API-COR-008 — Real-Time/Streaming Channel

Release Classification: Planned | **Governing Source:** No document specifies a protocol (see Appendix A Open Question #5); DS-MKT-003 (Committed) requires real-time/delayed distinction at the product level regardless of transport

Description: DarkSage should provide a real-time or near-real-time delivery mechanism for quotes, signals, Auto-Trader state, and notifications. The specific protocol (WebSocket, Server-Sent Events, or client polling) is not yet decided. Phase-1 Committed/MVP functionality (DS-MKT-003's real-time/delayed distinction) does not itself require a push-based channel — polling against DS-API-MKT-001 (§6.1) is sufficient to satisfy Phase-1 acceptance; a dedicated streaming channel is a Planned enhancement.

Acceptance Criteria: Deferred pending protocol decision.

Testing: Not yet applicable.

DS-API-COR-009 — API-Level Auditability

Release Classification: Committed / MVP | **Governing Source:** DS-OPS-001/002 (Committed)

Description: Every API request that materially changes state, or that reads a security-sensitive resource (credentials, risk configuration), shall be attributable to an authenticated caller and logged consistent with DS-OPS-001/002's AuditLogEntry (DS-DB-020) requirements.

Acceptance Criteria:

- A state-changing request without a resolvable authenticated caller is rejected, not logged as anonymous and allowed.
- Audit-relevant requests produce a correlated AuditLogEntry per DS-OPS-002.

Testing: Audit-correlation test: perform a state-changing request, confirm a matching AuditLogEntry exists.

DS-API-COR-010 — Session Lifecycle Contract

Release Classification: Committed / MVP | **Governing Source:** `SECURITY_RULES.md` "Authentication and Authorization"; DS-API-COR-004

Description: Authentication (DS-API-COR-004) requires a session lifecycle independent of the specific credential mechanism chosen (Appendix A Open Question #2 remains open — this requirement does not resolve JWT vs. opaque token vs. OAuth2). The API shall expose:

Operation	Method	Path	Description
Login	POST	`/auth/sessions`	Creates a new session from valid credentials; returns a session/access credential and its expiration.
Current session	GET	`/auth/sessions/current`	Returns metadata for the calling session itself (see below) — the discovery point for whether refresh is supported.
Logout	DELETE	`/auth/sessions/current`	Invalidates the calling session's credential immediately. Distinct operation from `GET` on the same path: `GET` reads current-session metadata and has no side effect; `DELETE` revokes it.
Refresh	POST	`/auth/sessions/refresh`	Renews an active session's credential before expiration, where the chosen mechanism supports renewal. A mechanism issuing only short-lived non-renewable credentials MAY omit this operation, but renewability MUST be discoverable from `GET /auth/sessions/current`.
List sessions	GET	`/auth/sessions`	Lists the caller's own active sessions/devices: `session_id`, `created_at`, `last_used_at`, device/client label, current-session flag.
Revoke one	DELETE	`/auth/sessions/{session_id}`	Revokes one specific session (e.g., a lost device), which may or may not be the caller's current session.
Revoke all	DELETE	`/auth/sessions`	Revokes every session for the caller, including the current one ("log out everywhere").

Every issued session credential has a defined, backend-enforced expiration; an expired credential is rejected with 401 regardless of client-side state.

`GET /auth/sessions/current` contract:

- **Purpose:** Returns metadata for the session the caller is currently authenticated with, including whether it is renewable — the deterministic discovery point the Refresh operation above requires.
- **Auth:** Requires a valid, non-expired, non-revoked session credential (the same credential this operation describes); an unauthenticated or expired/revoked request receives 401 — this endpoint never falls back to an anonymous or default response.
- **Request parameters:** None (path, query, and body are all empty; the target session is always inferred from the caller's own credential, never from a parameter — a caller can never query a session other than its own through this operation).
- **Response schema (200):** `session_id`, `created_at`, `last_used_at`, `expires_at`, `device_label`, `is_current: true` (always true here — this operation only ever describes the caller's own session, distinguishing it from a `session_id` entry returned by `GET /auth/sessions`), `refresh_supported` (boolean — the mechanism-agnostic discovery flag Refresh's row above requires), `refresh_expires_at` (nullable timestamp, present only when `refresh_supported` is true).
- **Success status:** 200.
- **Error statuses:** 401 (missing, expired, or revoked credential).
- **Secret exposure:** The response never contains the raw session credential (token/cookie value) itself, in any form — only metadata about it, consistent with DS-API-COR-004's and DS-API-INT-001's secret-exposure boundary. This holds regardless of which credential mechanism (Appendix A Open Question #2) is eventually chosen.

Acceptance Criteria:

- A session created by `POST /auth/sessions` is retrievable in the caller's own `GET /auth/sessions` list.
- `GET /auth/sessions/current` deterministically reports `refresh_supported` for the calling session, regardless of the underlying credential mechanism; a client can always determine renewability from this single call without trial-and-error against the Refresh operation.
- `GET /auth/sessions/current` and `DELETE /auth/sessions/current` never share side effects: repeated `GET` calls are idempotent and never revoke the session; only `DELETE` does.
- Revoking a session, individually or via revoke-all, causes subsequent requests bearing that session's credential to receive 401 without requiring a backend restart or cache expiry, including subsequent calls to `GET /auth/sessions/current` itself.
- An expired, unrevoked credential is rejected identically to a revoked one (401) from the caller's perspective, including on `GET /auth/sessions/current`.
- No operation in this contract requires the caller to know whether the underlying credential is JWT or opaque — the contract is mechanism-agnostic.
- No response in this contract, including `GET /auth/sessions/current`, ever exposes the raw session credential value.
- High-risk actions requiring the strong-authentication tier (DS-API-COR-004) MAY require a freshly-created or re-verified session distinct from a merely non-expired one; the exact distinction is governed by `SECURITY_RULES.md`, not redefined here.

Generated from DS-006 v0.2.1 — baseline 34a65818d48f596122281075e18c6e5f36ec93b5

Testing: Session-lifecycle test matrix: create → read current (assert `refresh_supported` present and correct) → list → use → refresh (where supported) → revoke-one → revoke-all → expire, confirming 401 on `GET /auth/sessions/current` and every other operation after each terminal state; secret-exposure scan across this operation's response.

6. API Domains

6.1 Market Data & Symbol API

Governing DS-002 family: DS-MKT (mostly Committed), DS-DAT-003 (Committed). Governing DS-005 entities: Candle (DS-DB-001), Quote (DS-DB-002), SecurityIdentity (DS-DB-014).

DS-API-MKT-001 — Quote and Candle Retrieval

Release Classification: Committed / MVP

Operations:

Operation	Method	Path	Path Params	Query Params	Response Body	Success	Errors
Quote	GET	`/symbols/{symbol_id}/quote`	`symbol_id`	—	current Quote (DS-DB-002) with `data_state`, `delay_seconds`	200	404 (unknown `symbol_id`), 401/403
Candles	GET	`/symbols/{symbol_id}/candles`	`symbol_id`	`timeframe` (required; enum: `1m`,`5m`,`15m`,`1h`,`1d` — exact supported set fixed at implementation, not expanded here), `start` (ISO-8601, optional), `end` (ISO-8601, optional, defaults to now), `limit` (optional, paginated per DS-API-COR-005), `adjusted` (optional bool, defaults to `true`)	Candle (DS-DB-001) array with `is_adjusted`, `data_state`, pagination metadata	200	400 (invalid `timeframe`, or `start` > `end`), 404 (unknown `symbol_id`), 401/403

Description: Returns the current Quote and historical Candle series for a resolved SecurityIdentity, with `data_state`/timestamp per DS-API-COR-002 and the staleness thresholds DS-MKT-004 defines. `timeframe` selects the candle granularity; `start`/`end` bound the requested range; a request with no `start` returns the most recent `limit` candles at that `timeframe` ending at `end`.

Auth: Authenticated (read-only permission group).

Acceptance Criteria: Response reflects the exact `data_state` and `delay_seconds`/`is_adjusted` fields DS-DB-001/002 define; a candle range request extending beyond available history returns the available subset plus an explicit `range_truncated`/`earliest_available` indicator (DS-MKT-002) rather than fabricating data or silently returning fewer candles with no explanation.

Testing: Fixture-based response-shape and staleness-labeling test; out-of-range request test asserting the truncation indicator.

DS-API-MKT-002 — Symbol Lookup and Resolution

Release Classification: Committed / MVP | **Endpoint(s):** `GET /symbols/search`, `GET /symbols/{symbol_id}`

Description: Resolves a ticker/search term to a canonical SecurityIdentity (DS-DB-014), and returns identity detail including `current_ticker` and `delisted_at`.

Auth: Authenticated (read-only).

Acceptance Criteria: A delisted-and-reused ticker search disambiguates per DS-DAT-003's edge case (does not silently merge two identities).

Testing: Ticker-reuse disambiguation test (shared with DS-DAT-003's own test).

DS-API-MKT-003 — Market Calendar

Release Classification: Committed / MVP | **Endpoint(s):** `GET /markets/{market}/calendar`

Description: Returns session state (pre-market/regular/post-market/closed/holiday) per DS-MKT-005, for whatever markets are supported (scope pending the Owner Decision recorded in DS-002 Appendix A).

Auth: Authenticated (read-only).

Testing: Session-state query test across regular/holiday/early-close fixtures (shared with DS-MKT-005's own test).

DS-API-MKT-004 — Watchlists

Release Classification: Planned | **Operations:** `GET /watchlists` (list, collection), `POST /watchlists` (create, collection), `GET /watchlists/{watchlist_id}` (item), `DELETE /watchlists/{watchlist_id}` (item), `POST /watchlists/{watchlist_id}/items` (add item, body: `symbol_id`), `DELETE /watchlists/{watchlist_id}/items/{symbol_id}` (remove item, path identifiers only).

Description: CRUD for Watchlist/WatchlistItem (DS-DB-018), per DS-MKT-007 (Planned). `watchlist_id` is Watchlist's own primary key; a WatchlistItem is identified by the composite of its parent `watchlist_id` and member `symbol_id`.

Auth: Authenticated (owner-scoped).

Testing: Deferred to DS-MKT-007's own implementation timing.

6.2 Scanner API

Governing DS-002 family: DS-SCN (Committed). Governing DS-005 entities: ScanConfiguration (DS-DB-021), ScanResult (DS-DB-022, Planned).

DS-API-SCN-001 — Scan Configuration CRUD

Release Classification: Committed / MVP

Operations:

Operation	Method	Path	Path Params	Query Params	Request Body	Response Body	Success	Errors
List	GET	`/scans`	—	`profile_id?` (filter)	—	array of ScanConfiguration summaries + pagination (DS-API-COR-005)	200	401/403
Create	POST	`/scans`	—	—	ScanConfiguration fields (`name`, `universe`, `filters`, `ranking`, optional `profile_id`)	created ScanConfiguration	201	400 (invalid filter/ranking definition), 401/403
Get	GET	`/scans/{scan_id}`	`scan_id`	—	—	ScanConfiguration	200	404, 401/403
Update	PUT	`/scans/{scan_id}`	`scan_id`	—	full ScanConfiguration fields (replace)	updated ScanConfiguration	200	404, 400, 401/403
Delete	DELETE	`/scans/{scan_id}`	`scan_id`	—	—	—	204	404, 401/403

Identifier: `scan_id` is ScanConfiguration's (DS-DB-021) own primary key. `profile_id` is nullable (resolves to the implicit default local profile per the DS-005-A03 repair) and is never itself a valid identifier for these operations.

Description: CRUD for ScanConfiguration (DS-DB-021).

Auth: Authenticated (owner-scoped, or unscoped for the default profile).

Acceptance Criteria: Deleting a ScanConfiguration (`DELETE /scans/{scan_id}`) does not delete its historical ScanResults (DS-SCN-005's immutability requirement); those remain retrievable via DS-API-SCN-003 by the now-orphaned `scan_id` reference.

Testing: CRUD regression test; orphaned-result-retention test.

DS-API-SCN-002 — Scan Execution

Release Classification: Committed / MVP | **Endpoint(s):** `POST /scans/{id}/execute`

Description: Executes a ScanConfiguration's deterministic pre-filtering and ranking (DS-SCN-001/002/003) against its universe, returning matched Signals (§6.3).

Auth: Authenticated.

Acceptance Criteria: Response discloses which symbols were evaluated and why each passed/failed, per DS-SCN-001's inspectability requirement.

Testing: Empty-universe test; filter-explanation completeness test.

DS-API-SCN-003 — Scan Result History

Release Classification: Planned | **Endpoint(s):** `GET /scans/{id}/results`

Description: Paginated historical ScanResult (DS-DB-022) retrieval, per DS-SCN-005 (Planned).

Auth: Authenticated.

Testing: Deferred to DS-SCN-005's own implementation timing.

6.3 Signal API

Governing DS-002 family: DS-SIG (Committed core). Governing DS-005 entity: Signal (DS-DB-004).

DS-API-SIG-001 — Signal List and Detail

Release Classification: Committed / MVP | **Endpoint(s):** `GET /signals`, `GET /signals/{signal_id}`

Description: Returns Signal (DS-DB-004) records with grade, scores, detected patterns, and reasoning, per DS-SIG-001/002.

Auth: Authenticated (read-only).

Acceptance Criteria: Grade and scores match DS-SIG-002's deterministic derivation; response never presents an AI-judgment-only grade.

Testing: Grade-derivation determinism test (shared with DS-SIG-002's own test).

DS-API-SIG-002 — Why-Trade / Why-Not-Trade Detail

Release Classification: Committed / MVP | **Endpoint(s):** `GET /signals/{signal_id}/reasons`

Description: Returns the machine-readable rejection/acceptance reason set DS-SIG-003 requires, drawn from its defined vocabulary.

Auth: Authenticated (read-only).

Testing: Rejection-reason completeness and vocabulary-conformance test (shared with DS-SIG-003's own test).

6.4 Chart API

Governing DS-002 family: DS-CHT (Committed core). Governing DS-005 entity: ChartAnnotation (DS-DB-013, Planned).

DS-API-CHT-001 — Chart Data

Release Classification: Committed / MVP | **Endpoint(s):** `GET /symbols/{symbol_id}/chart`

Description: Returns candle series plus computed indicator values (DS-CHT-001/002) for a requested timeframe/range, engine-agnostic per DS-ARC-008 (both ECharts and TradingView Lightweight Charts consume the same response).

Auth: Authenticated (read-only).

Acceptance Criteria: Indicator values are identical regardless of which chart engine the client renders with (DS-ARC-008's cross-renderer parity, at the data layer).

Testing: Cross-renderer data-parity test (shared with DS-ARC-008's own test).

DS-API-CHT-002 — Chart Annotations

Release Classification: Planned | **Operations:** `GET /symbols/{symbol_id}/annotations` (list, collection, query `timeframe?`), `POST /symbols/{symbol_id}/annotations` (create, collection, body: annotation shape/position/text), `DELETE /symbols/{symbol_id}/annotations/{annotation_id}` (item).

Description: CRUD for ChartAnnotation (DS-DB-013), per DS-CHT-003 (Planned). `annotation_id` is ChartAnnotation's own primary key, scoped to its parent `symbol_id`.

Auth: Authenticated (owner-scoped).

Testing: Deferred to DS-CHT-003's own implementation timing.

6.5 Workspace API

Governing DS-002 family: DS-WKS-001 (baseline Committed); DS-WKS-002/003 (Planned). Governing DS-005 entity: WorkspaceLayout (DS-DB-017, Planned overall — its Release Classification note that "baseline shell is Committed" describes DS-WKS-001's product-level obligation, not a Committed status for the entity itself). Per the DS-006-H2 repair, the Committed/MVP endpoint below does not depend on DS-DB-017; only the Planned persisted-layout endpoint does.

DS-API-WKS-001 — Active Workspace Composition

Release Classification: Committed / MVP | **Endpoint(s):** `GET /workspace/layout/current`, `PUT /workspace/layout/current`

Description: Reads/writes the active, in-session workspace's widget composition (add/remove/resize/arrange), per DS-WKS-001's baseline obligation. This Committed/MVP behavior is backed by ephemeral, session-scoped runtime state on the backend (consistent with DS-ARC-001's backend-authoritative model) and does not require the persisted WorkspaceLayout entity (DS-DB-017), which remains Planned. Before durable persistence exists (DS-WKS-003/DS-API-WKS-002, both Planned), a restart or reconnect MAY lose the active composition and fall back to a default view — this is the exact behavior DS-PRD-010 (Committed) already anticipates: "restoring prior workspace state where layout persistence exists (DS-WKS-003, Planned pending that feature's implementation), or presenting a default view otherwise."

Auth: Authenticated (owner-scoped).

Acceptance Criteria:

- Writing a layout referencing an unknown widget type is rejected with a specific error (DS-API-COR-006), not silently accepted.
- This endpoint's Committed/MVP status never implies durable cross-session persistence; that capability is exclusively DS-API-WKS-002 (Planned). No test or integration may treat a restart-survival failure here as a Committed/MVP regression.

Testing: Baseline layout read/write test, scoped to a single session (no restart-persistence assertion).

DS-API-WKS-002 — Persisted and Saved Workspace Layouts

Release Classification: Planned | **Endpoint(s):**

Operation	Method	Path	Path Params	Notes
List	GET	`/workspace/layouts`	—	Named, saved WorkspaceLayouts (DS-DB-017) for the caller.

Operation	Method	Path	Path Params	Notes
Create	POST	`/workspace/layouts`	—	Persists a new named layout, optionally from the current in-session composition (DS-API-WKS-001).
Get	GET	`/workspace/layouts/{layout_id}`	`layout_id`	—
Update	PUT	`/workspace/layouts/{layout_id}`	`layout_id`	Replaces the named layout's composition.
Delete	DELETE	`/workspace/layouts/{layout_id}`	`layout_id`	—
Activate	POST	`/workspace/layouts/{layout_id}/activate`	`layout_id`	Makes this saved layout the active composition served by DS-API-WKS-001.

Description: CRUD and activation for multiple named, persisted WorkspaceLayouts (DS-DB-017), per DS-WKS-003. This is the sole owner of durable layout persistence; once implemented, it supersedes DS-API-WKS-001's ephemeral backing store for the "current" composition (the active layout becomes whichever named layout was last activated, or an unsaved session-only composition if none has been).

Auth: Authenticated (owner-scoped).

Testing: Deferred to DS-WKS-003's own implementation timing.

6.6 Integration and Credential API

Governing DS-002 family: DS-INT-002, DS-SEC-001 (Committed). Governing DS-005 entity: IntegrationCredential (DS-DB-019).

DS-API-INT-001 — Integration Credential Management

Release Classification: Committed / MVP | **Endpoint(s):** `GET/POST/PUT/DELETE /integrations/credentials`

Description: CRUD for IntegrationCredential (DS-DB-019). Create/update accepts a raw secret value for one-time secure storage; every read response returns only `secure_storage_reference`-derived metadata (`status`, `provider_name`, `last_validated_at`) and never a usable secret, per DS-SEC-001/DS-DB-019's constraint.

Auth: Authenticated (administrative settings permission group per `SECURITY_RULES.md`).

Operations:

Operation	Method	Path	Path Params	Request Body	Response Body	Success	Errors
List	GET	`/integrations/credentials`	—	—	array of credential metadata (`credential_id`, `provider_name`, `status`, `last_validated_at`)	200	401/403

Generated from DS-006 v0.2.1 — baseline 34a65818d48f596122281075e18c6e5f36ec93b5

Operation	Method	Path	Path Params	Request Body	Response Body	Success	Errors
Create	POST	`/integrations/credentials`	—	`provider_name`, raw secret value, provider-specific fields	created credential metadata (never the secret)	201	400 (invalid provider/secret shape), 401/403, 409 (credential already exists for this provider)
Get	GET	`/integrations/credentials/{credential_id}`	`credential_id`	—	credential metadata	200	404, 401/403
Update	PUT	`/integrations/credentials/{credential_id}`	`credential_id`	raw secret value and/or provider-specific fields to replace	updated credential metadata	200	404, 400, 401/403
Delete	DELETE	`/integrations/credentials/{credential_id}`	`credential_id`	—	—	204	404, 401/403, 409 (credential in use by an active integration that requires one)

Identifier: `credential_id` is IntegrationCredential's (DS-DB-019) own primary key, never the raw secret or `secure_storage_reference`.

Acceptance Criteria: No response, log, or error message from this endpoint group ever contains a raw secret value (DS-SEC-001).

Testing: Secret-exposure scan across request/response/log/error paths for this endpoint group (shared with DS-SEC-001's own test).

6.7 User and Preferences API

Governing DS-002 family: DS-USR (Planned). Governing DS-005 entities: UserProfile (DS-DB-015), Preferences (DS-DB-016).

DS-API-USR-001 — Profile and Preferences

Release Classification: Planned | **Operations:** `GET /profile` (item, current caller's UserProfile), `PUT /profile` (item, replace UserProfile fields), `GET /profile/preferences` (item), `PUT /profile/preferences` (item, replace Preferences fields).

Description: Reads/writes UserProfile (DS-DB-015) and Preferences (DS-DB-016) — terminology mode, notification settings — per DS-USR-002/003/006 (all Planned). Both resources are singleton per caller; no separate identifier is needed beyond the authenticated session.

Auth: Authenticated (owner-scoped).

Testing: Deferred to DS-USR-002/003/006's own implementation timing.

6.8 Sage/AI API

Governing DS-002 family: DS-AI (Planned, Phase 6). Governing DS-003: DS-SGE family (Planned, Phase 6). No dedicated DS-005 entity (conversational state is session-scoped per DS-SGE-010, not necessarily persisted to a DS-005 entity in Phase 6's initial scope).

DS-API-AI-001 — Sage Conversational Interface

Release Classification: Planned (Phase 6) | **Endpoint(s):** `POST /sage/messages` (or a streaming variant per DS-API-COR-008 once decided)

Description: Sends a user message to Sage and returns a response grounded in enabled evidence, per DS-AI-001. Response payload distinguishes recommendation language from confirmed-action language (DS-PRD-005, unconditionally Committed) and applies the confidence vocabulary DS-SGE-014 defines.

Auth: Authenticated.

Acceptance Criteria: This endpoint's response schema can never itself trigger a state-changing action — any action Sage recommends still requires a separate, explicit confirmation call to the relevant domain endpoint (DS-PRD-005, DS-SGE-005), per the Core Rule in DS-003.

Testing: Deferred to DS-AI-001/DS-SGE's own implementation timing; the no-implicit-execution acceptance criterion is testable now as a contract constraint on this endpoint's design.

DS-API-AI-002 — Evidence and Explanation Retrieval

Release Classification: Planned (Phase 6) | **Endpoint(s):** `GET /sage/messages/{id}/evidence`

Description: Returns the cited evidence/sources and explainability detail (DS-AI-003, DS-SGE-008/013) behind a given Sage response.

Auth: Authenticated.

Testing: Deferred to DS-AI-003/DS-SGE-013's own implementation timing.

6.9 Strategy API

Governing DS-002 family: DS-STR (Planned, Phase 2). Governing DS-005 entity: StrategyProfile (DS-DB-005, Committed Phase-1 core / Planned Phase-2 extension).

DS-API-STR-001 — Strategy CRUD and Versioning

Release Classification: Committed / MVP for the Phase-1 core fields only (id, name, version, status); Planned for the full construction feature (configuration, risk_assumptions) | **Endpoint(s):** `GET /strategies`, `GET /strategies/{id}`, `POST /strategies` (Planned — full construction), `PUT /strategies/{id}` (Planned — creates a new version, never mutates `configuration` in place per `AGENTS.md`)

Description: Exposes StrategyProfile per DS-DB-005's field split: read access to the Phase-1 core is Committed (supports Signal's optional `strategy_id` reference even before strategy construction exists); the create/update surface enabling actual strategy authoring is Planned pending DS-STR-001's implementation.

Auth: Authenticated (owner-scoped for create/update).

Acceptance Criteria: A `PUT` never overwrites a prior version's `configuration`; it creates a new versioned row (DS-DB-005's `superseded_by` chain).

Testing: Version-chain integrity test (shared with DS-DB-005's own test).

6.10 Backtest API

Governing DS-002 family: DS-BKT (Planned, Phase 2). Governing DS-005 entity: BacktestResult (DS-DB-006, Planned).

DS-API-BKT-001 — Backtest Execution and Results

Release Classification: Planned | **Endpoint(s):** `POST /backtests`, `GET /backtests/{id}`

Description: Executes a strategy-version-pinned backtest and returns its immutable BacktestResult (DS-DB-006), per DS-BKT-001/002/003/004.

Auth: Authenticated.

Acceptance Criteria: Response always includes the DS-BKT-004 disclosure that historical/simulated performance is not a guarantee of future performance.

Testing: Deferred to DS-BKT's own implementation timing; disclosure-presence is testable as a response-schema contract now.

6.11 Strategy Performance API

Governing DS-002 family: DS-PERF (Planned, Phase 3). No new DS-005 entity (uses MarketRegime, BacktestResult).

DS-API-PERF-001 — Performance Metrics and Strategy DNA

Release Classification: Planned | **Endpoint(s):** `GET /strategies/{id}/performance`, `GET /symbols/{symbol_id}/strategy-dna`

Description: Returns segmented performance metrics (DS-PERF-001/002) and per-symbol Strategy DNA (DS-PERF-003), statistically derived, never AI-guessed.

Auth: Authenticated.

Testing: Deferred to DS-PERF's own implementation timing.

6.12 Risk API

Governing DS-002 family: DS-RSK-001 (Committed authority boundary); DS-RSK-002/003/005 (Planned). Governing DS-005 entity: RiskState (DS-DB-011, Planned).

DS-API-RSK-001 — Risk Engine Query (Read-Only)

Release Classification: Committed / MVP (query boundary) | **Endpoint(s):** `GET /risk/state` (Planned — full RiskState detail, gated behind DS-DB-011's own Planned classification)

Description: Establishes, at the API-contract level, that any Risk Engine query surface is strictly read-only for callers — no request body on this endpoint group can alter a risk rule or determination (DS-RSK-001, DS-PRD-006, unconditionally Committed). The actual RiskState payload (DS-DB-011) is Planned pending Portfolio's own Phase-4 timing.

Auth: Authenticated.

Acceptance Criteria: No endpoint in this group accepts a request body capable of modifying a Risk Engine rule; rule changes are exposed (once they exist) only through a distinct, administratively-gated risk-configuration surface, never this query group.

Testing: Adversarial test confirming no risk-mutation path exists behind a read-labeled endpoint (mirrors DS-RSK-001's own adversarial test).

6.13 Portfolio API

Governing DS-002 family: DS-PRT (Planned, Phase 4). Governing DS-005 entities: Position (DS-DB-009), Portfolio (DS-DB-008), **Transaction (DS-DB-025, added in the DS-005-A01 repair)**.

DS-API-PRT-001 — Positions and Portfolio Summary

Release Classification: Planned | **Endpoint(s):** `GET /portfolios/{id}`, `GET /portfolios/{id}/positions`

Description: Returns Portfolio (DS-DB-008) aggregate state and its Position (DS-DB-009) list, mark-to-market against current/last-known Quote per DS-PRD-008.

Auth: Authenticated (owner-scoped).

Testing: Deferred to DS-PRT's own implementation timing.

DS-API-PRT-002 — Transaction Recording and History

Release Classification: Planned | **Endpoint(s):** `GET /portfolios/{id}/transactions`, `POST /portfolios/{id}/transactions`

Description: Records and retrieves Transaction (DS-DB-025) entries — the append-only ledger Position and realized/unrealized performance derive from. A `POST` creates a new, immutable Transaction; correcting a prior entry requires a new Transaction with `reverses_transaction_id` set, never an update to the original (DS-PRT-002, DS-DB-023).

Auth: Authenticated (owner-scoped).

Acceptance Criteria: No endpoint in this group exposes an update or delete operation against a confirmed Transaction — only create (including reversing entries).

Testing: Append-only enforcement test (shared with DS-DB-025's own test); reversal-chain integrity test.

6.14 Execution, Auto-Trader, and Broker API

Governing DS-002 family: DS-EXE (governance boundary Committed; implementation Planned, Phase 7); DS-EXE-002 (Trade Proposal Representation, Planned). Governing DS-005 entities: TradeDecision (DS-DB-007, Committed Phase-1 core / Planned extension), Order (DS-DB-010, Planned), BrokerState (DS-DB-012, Planned per the DS-005-A03 repair).

DS-API-EXE-001 — TradeValidationPipeline Boundary (Governance Contract)

Release Classification: Committed / MVP | **Endpoint(s):** N/A — this requirement constrains every endpoint in this domain, not a single endpoint.

Description: No endpoint in this API, now or in any future version, may accept a request that would submit an order or otherwise reach the Broker Adapter without first passing the full canonical `TradeValidationPipeline` (`ARCHITECTURE.md` §14, DS-ARC-011, DS-EXE-001). This is the API-layer restatement of DS-EXE-001's unconditional product-level boundary and ADR-002.

Acceptance Criteria: Every write endpoint added to this domain in a future revision is reviewed against this requirement before merge; no endpoint bypasses Signal Validator → Strategy Validation → Risk Engine → Permissions Engine → Portfolio/Exposure → Buying Power → Market Condition → Order Validation → Execution Engine → Broker Adapter.

Testing: Requirements review for every future addition to this domain (mirrors DS-EXE-001's own test).

DS-API-EXE-002 — Trade Proposal Read Access

Release Classification: Committed / MVP (Phase-1 core read access only) | **Endpoint(s):** `GET /trade-decisions`, `GET /trade-decisions/{id}`

Description: Read-only access to TradeDecision (DS-DB-007) records at their Phase-1 core level (decision_id, signal_id, strategy_id, proposed_size/direction, final_disposition □ {proposed, rejected}). Pipeline/AI extension fields (`pipeline_stage_results`, `ai_provider_used`, `ai_output_raw`) are included in the response only once populated (Phase 6/7), per DS-DB-007's field split.

Auth: Authenticated (read-only).

Testing: Phase-1-core response-shape test with no extension fields populated (shared with DS-DB-007's own test).

DS-API-EXE-003 — Auto-Trader Control and Emergency Stop/Flatten

Release Classification: Planned | **Operations:** `GET /auto-trader/state` (item, singleton), `PUT /auto-trader/state` (item, body: target state □ {disabled, enabled, paused}; `emergency_stop` is never a valid target of this operation — only reachable via the dedicated endpoint below), `POST /auto-trader/emergency-stop` (no body, sets state to `emergency_stop`), `POST /auto-trader/emergency-flatten` (no body beyond required strong-auth credential in live mode).

Description: Reads/controls Auto-Trader state (disabled/enabled/paused/emergency_stop, DS-EXE-003) and triggers Emergency Stop/Flatten (DS-EXE-004/005), reachable from any authorized client independent of the normal order-submission path (DS-ARC-022).

Auth: Authenticated; Emergency Flatten requires the strong-authentication tier in live mode (`SECURITY_RULES.md`).

Acceptance Criteria: Emergency Stop/Flatten endpoints remain reachable even if the normal trade-submission code path is degraded (DS-ARC-022's independent-reachability requirement).

Testing: Deferred to Phase 7 implementation; independent-reachability is testable as an architectural contract now.

DS-API-EXE-004 — Order and Broker State (Read)

Release Classification: Planned | **Endpoint(s):** `GET /orders`, `GET /orders/{id}`, `GET /portfolios/{id}/broker-state`

Description: Read access to Order (DS-DB-010) and BrokerState (DS-DB-012) for reconciliation display, per DS-EXE-006/DS-ARC-021.

Auth: Authenticated (owner-scoped).

Testing: Deferred to Phase 7 implementation.

DS-API-EXE-005 — Trade Proposal Creation

Release Classification: Planned | **Endpoint(s):** `POST /trade-proposals`

Description: Creates a Trade Proposal (DS-EXE-002) as a structured, advisory-only object (`signal_id`, `strategy_id`, `proposed_size`, `proposed_direction`), whether authored by Sage or directly by the user. Creating a proposal through this endpoint never submits an order, never reaches the Broker Adapter, and never invokes any `TradeValidationPipeline` stage beyond structural validation of the proposal's own shape (DS-EXE-002). The created proposal is persisted

as a TradeDecision (DS-DB-007) with `final_disposition = proposed`.

Auth: Authenticated.

Acceptance Criteria:

- No request to this endpoint can result in an Order (DS-DB-010) being created or a Broker Adapter being invoked, directly or indirectly (DS-API-EXE-001 applies).
- A proposal created here is immediately visible via DS-API-EXE-002's read access with `final_disposition = proposed`.
- Sage-authored and user-authored proposals are indistinguishable in privilege at this endpoint — creation alone never confers execution authority ("Sage advises. The user decides.," DS-PRD-005).

Testing: Adversarial test confirming no code path from this endpoint reaches the Execution Engine or Broker Adapter; proposal-shape validation test.

DS-API-EXE-006 — Trade Proposal Submission into the TradeValidationPipeline

Release Classification: Planned | **Endpoint(s):** `POST /trade-proposals/{proposal_id}/submit`

Description: The only mechanism by which an existing Trade Proposal (DS-API-EXE-005) may proceed toward execution. Requires an explicit, separate, authenticated user confirmation distinct from proposal creation — no Sage action and no prior API call may itself constitute this confirmation (DS-PRD-005, DS-SGE-005). On submission, the proposal enters the canonical `TradeValidationPipeline` (`ARCHITECTURE.md` §14) at its first stage and proceeds through every stage in full and in order (DS-EXE-001, DS-API-EXE-001); this endpoint performs no validation itself and makes no execution decision — it only triggers pipeline entry. A proposal that fails any stage is rejected with the failing stage and reason in the response (DS-API-COR-006 error schema), and `final_disposition` updates accordingly (`validated`/`rejected`/`executed`, per DS-DB-007's Phase-6/7 extension). Order placement, if any, occurs only as the terminal outcome of a fully-passed pipeline via the Execution Engine → Broker Adapter (DS-EXE-006); live-mode order placement remains blocked for every caller, including Sage-originated proposals, until DS-EXE-007's live-trading gate is satisfied — paper-mode submission remains available before then.

Auth: Authenticated; live-mode submission additionally requires the strong-authentication tier once DS-EXE-007's gate is satisfied (`SECURITY_RULES.md`).

Acceptance Criteria:

- No implementation of this endpoint may skip, reorder, or short-circuit any `TradeValidationPipeline` stage (DS-API-EXE-001 applies).
- A proposal cannot self-submit — creation (DS-API-EXE-005) and submission are always two distinct, separately authorized calls.
- Live-mode order placement via this path remains blocked for all callers until DS-EXE-007's gate is satisfied; paper-mode submission is unaffected by that gate.
- Rejection at any stage returns the failing stage and reason; it never returns a generic/opaque failure.

Testing: Full pipeline-stage-order adversarial test (mirrors DS-API-EXE-001's own test); paper-vs-live gating test tied to DS-EXE-007.

6.15 Alerts and Notifications API

Governing DS-002 family: DS-ALT (Planned). Governing DS-005 entities: **Alert (DS-DB-026) and Notification (DS-DB-027), both added in the DS-005-A02 repair.**

DS-API-ALT-001 — Alert Configuration

Release Classification: Planned

Operations:

Operation	Method	Path	Path Params	Query Params	Request Body	Response Body	Success	Errors
List	GET	`/alerts`	—	`profile_id?` (filter)	—	array of Alert summaries + pagination	200	401/403
Create	POST	`/alerts`	—	—	Alert fields (`condition_definition`, optional `profile_id`)	created Alert	201	400 (invalid `condition_definition`), 401/403
Get	GET	`/alerts/{alert_id}`	`alert_id`	—	—	Alert	200	404, 401/403
Update	PUT	`/alerts/{alert_id}`	`alert_id`	—	full Alert fields (replace)	updated Alert	200	404, 400, 401/403
Delete	DELETE	`/alerts/{alert_id}`	`alert_id`	—	—	—	204	404, 401/403

Identifier: `alert_id` is Alert's (DS-DB-026) own primary key.

Description: CRUD for Alert (DS-DB-026), per DS-ALT-001. Alert `condition_definition` may reference a symbol/indicator/threshold or a risk-limit condition.

Auth: Authenticated (owner-scoped, or default-profile-scoped per DS-DB-026's nullable `profile_id` pattern).

Acceptance Criteria: No operation in this table — create, update, or any other — can create an Order or reach the Execution/Broker domain (§6.14); no request body field on any operation is capable of specifying broker or order data. An Alert firing produces only a Notification (DS-PRD-007's notification-only boundary, unconditionally Committed).

Testing: Notification-only boundary adversarial test (shared with DS-DB-026's own constraint).

DS-API-ALT-002 — Notification History and Delivery

Release Classification: Planned

Operations:

Operation	Method	Path	Path Params	Query Params	Request Body	Response Body	Success	Errors
List	GET	`/notifications`	—	`since?` (ISO-8601, filter), `unread_only?` (bool), `limit`/pagination (DS-API-COR-005)	—	array of Notification (DS-DB-027) + pagination	200	401/403
Get	GET	`/notifications/{notification_id}`	`notification_id`	—	—	Notification	200	404, 401/403
Mark read	PUT	`/notifications/{notification_id}/read`	`notification_id`	—	—	updated Notification (`read_at` set)	200	404, 401/403

Identifier: `notification_id` is Notification's (DS-DB-027) own primary key.

Description: Retrieves Notification history — including notifications fired while the client was closed/backgrounded, per DS-ALT-002's restart-recovery requirement — and marks a notification read/acknowledged.

Auth: Authenticated (owner-scoped).

Acceptance Criteria: A Notification persisted before the client last connected is retrievable in full via `GET /notifications` (DS-ALT-002's restart-recovery edge case), filterable by `since` to fetch only what the client has not yet seen.

Testing: Restart-recovery test (shared with DS-DB-027's own test).

6.16 Education and Knowledge API

Governing DS-002 family: DS-EDU (Planned/Future). No dedicated DS-005 entity yet.

DS-API-EDU-001 — Contextual Terminology Lookup

Release Classification: Planned | **Endpoint(s):** `GET /terms/{term}`

Description: Returns a definition/explanation for a domain term referenced elsewhere in the product, per DS-EDU-001, respecting the active terminology mode (DS-USR-003) once that feature exists.

Auth: Authenticated (read-only).

Testing: Deferred to DS-EDU-001's own implementation timing.

6.17 Audit and Observability API

Governing DS-002 family: DS-OPS-001/002 (Committed). Governing DS-005 entity: AuditLogEntry (DS-DB-020).

DS-API-OPS-001 — Audit Log Query

Release Classification: Committed / MVP | **Endpoint(s):** `GET /audit-log`

Description: Read access to AuditLogEntry (DS-DB-020) for local transparency/diagnosis, per DS-OPS-001. Response `detail` is pre-redacted of secrets at write time (DS-DB-020's own constraint); as defense-in-depth against a future write-time redaction defect, this endpoint additionally applies a response-time secret-pattern scan before returning `detail`, redacting any match rather than relying solely on write-time correctness.

Auth: Authenticated (administrative settings permission group).

Acceptance Criteria: No response from this endpoint ever contains a raw secret value (shared boundary with DS-API-INT-001/DS-SEC-001).

Testing: Secret-exposure scan across this endpoint's responses (shared with DS-DB-020's own test).

7. Mobile API Contract Considerations

Per `ROADMAP.md` Phase 1's explicit inclusion of "Mobile API contracts" as a Phase 1 backend deliverable — even though the Mobile app itself is Phase 9 (DS-MOB, DS-ARC-003) — every

Committed/MVP endpoint in §6 shall be designed mobile-ready from the start: stable, versioned (DS-API-COR-003), and independent of any desktop-specific assumption (e.g., no endpoint may assume a persistent local desktop process). This is a design constraint on the Committed/MVP endpoints authored now, not a deferred Phase-9 concern. Mobile-specific endpoints (offline cached read-only snapshots, push-notification registration, trade-approval flows) are Planned, matching DS-MOB-001/DS-ARC-003's own classification.

8. Non-Goals

DS-006 does not: select the specific API versioning scheme, authentication scheme, pagination style, or real-time transport protocol (all recorded as Appendix A Open Questions, not invented here); redesign any DS-004 architectural boundary; alter any DS-005 entity schema; or commit endpoints for Planned/Future DS-002 requirements beyond stating their prospective contract shape.

9. Dependencies

- DS-001, DS-002, DS-003, DS-004, DS-005
- `ARCHITECTURE.md` §2, §14; `SECURITY_RULES.md`; `TRADING_RULES.md`; `ROADMAP.md` Phase 1
- `.ai-workflow/DS-006-PREP-NOTES.md` (local preparation input, not a controlled source)

10. Risks and Constraints

- **Five cross-cutting decisions remain open** (versioning scheme, auth scheme, pagination style, rate-limit policy, real-time protocol) — each stated as an obligation now (something must exist) without a chosen mechanism, consistent with the DS-002/DS-004 "WHAT vs. HOW" pattern (DS-002-H04/A04's resolution). None block this draft's completeness; all are recorded in Appendix A.
- **Sequencing risk:** authored after DS-005's Transaction/Alert/Notification additions, so the Portfolio and Alerts domains did not need to be stubbed pending-entity, unlike what the DS-006 preparation notes anticipated before that repair landed.
- **Classification discipline:** applied the same Committed/MVP eligibility test used throughout DS-002/DS-004/DS-005 — an endpoint's classification matches its underlying DS-002 requirement's or DS-005 entity's classification; a Committed endpoint group's read-only core (e.g., DS-API-EXE-002, DS-API-STR-001) is separated from its Planned write/full-feature surface using the same "Phase-1 core vs. later-phase extension" pattern established for `DS-ARC-005`/`DS-DB-007`.

11. Verification Approach

Each `DS-API-<DOMAIN>-NNN` requirement states its own Testing. Document-level verification (unique-ID check, cross-reference consistency against DS-002/DS-004/DS-005, no contradiction with DS-001/ADR-001–004, no Committed endpoint requiring Planned-only capability) recorded in `.ai-workflow/HANDOFF.md`.

12. References

- docs/codex/Volume-02-Product/DS-002-SRS.md and requirements/*.md
- docs/codex/Volume-04-Architecture/DS-004-Technical-Architecture.md
- docs/codex/Volume-05-Database/DS-005-Database-Design.md
- ARCHITECTURE.md, SECURITY_RULES.md, TRADING_RULES.md, ROADMAP.md

Appendix A — Open Questions

1. **API versioning scheme** — URL-path, header-based, or content-negotiation not yet decided (DS-API-COR-003). Routine implementation detail, not an owner-decision blocker.
2. **Authentication scheme** — JWT vs. opaque session token vs. OAuth2 not yet decided (DS-API-COR-004). SECURITY_RULES.md mandates authentication and session properties but not the concrete mechanism. The session lifecycle contract itself (login/logout/refresh/revocation/device visibility) is fixed and mechanism-agnostic per DS-API-COR-010 — only the credential format remains open.
3. **Pagination style** — cursor-based vs. offset/limit not yet decided (DS-API-COR-005).
4. **Rate-limiting policy** — no specific limits/windows defined yet (DS-API-COR-007).
5. **Real-time transport protocol** — WebSocket vs. Server-Sent Events vs. polling not yet decided (DS-API-COR-008); Phase-1 Committed/MVP acceptance does not require resolving this (polling against existing read endpoints suffices).
6. **Missing Auth/Session requirement family at the DS-002/DS-004 level** — DS-API-COR-004 is grounded directly in SECURITY_RULES.md since no DS-002/DS-004 requirement currently owns authentication/session behavior as a first-class product/architecture requirement. Recommend a future DS-002/DS-004 addition to formalize this rather than leaving DS-006 as its only home.
7. **Governance-confirmation carryover** — the standing BLOCKERS.md items (ROADMAP.md phase boundaries as Codex release-scope authority; phase-mapping precision) apply identically to this document's Release Classification scheme and are not re-litigated here.

Generated from DS-006 v0.2.1 — baseline 34a65818d48f596122281075e18c6e5f36ec93b5